Detailed C# Features for Learners

1. Anonymous Methods and Lambda Expressions

Category	Description
Definition	**Anonymous Method:** A method defined inline without a name, primarily used with delegates. **Lambda Expression:** A concise syntax for writing anonymous methods, introduced to simplify the code.
Features	- Anonymous methods use the `delegate` keyword and are typically longer in syntax. - Lambda expressions use the `=>` syntax and are considered a modern, short syntax. - Both are used to implement delegates like `Func`, `Action`, and `Predicate`.
Real-Life Example	They are widely used in Language Integrated Query (LINQ) statements for data filtering, sorting, and projection, where you need to pass a small piece of logic dynamically.
FAQ in Interviews	Q: What is a lambda expression? Give a syntax example. Q: How do you achieve cleaner and more readable code using lambda expressions?
Best Practices	Prefer lambda expressions (`=>`) over the older anonymous methods (`delegate` keyword) for concise, cleaner, and more readable code.
Application-Based Case	In a **Student Activity Evaluation System**, a lambda expression is used to check student eligibility: `Predicate<Student> IsEligible = student => student.Marks > 50;`.

2. Func, Action, and Predicate Delegates

Category	Description
Definition	Built-in delegate types provided by the .NET framework that eliminate the need to define custom delegate signatures for common operations.
Features	**- Func<T, TResult>**: Accepts one or more parameters and **returns a value**. **- Action<T>**: Accepts parameters but **has no return value** (void).

	- Predicate<T>: Accepts a single parameter and **returns a boolean value**.
Real-Life Example	- `Func`: Used for calculations, data transformations, and as arguments in LINQ methods. - `Action`: Used for logging, printing, or displaying data. - `Predicate`: Used for filtering data collections, such as in `List<T>.FindAll()`.
FAQ in Interviews	Q: What are `Func`, `Action`, and `Predicate` delegates? Q: When is it appropriate to use an `Action` versus a `Func`?
Best Practices	Use these built-in types to standardize delegate usage, which improves compatibility and makes code immediately understandable to other C# developers.
Application-Based Case	**Student Evaluation System:** A `Func<List<int>, int>` can be used with an anonymous method to calculate and return the total marks for a student.

3. Events

Category	Description
Definition	A special delegate that implements the publish-subscribe pattern, allowing a class (publisher) to notify other objects (subscribers) when something notable occurs.
Features	- Decouples the publisher from the subscriber, meaning the publisher does not need to know which methods will handle the notification. - The event handler is typically a method that matches the delegate's signature.
Real-Life Example	Used for user interface (UI) clicks, system notifications, and communication between components in decoupled architectures.
FAQ in Interviews	Q: What is a delegate and how does it differ from an event? Q: What design pattern do C# events follow?
Best Practices	Use the standard `EventHandler` or `EventHandler<TEventArgs>` types for defining events. Events should generally be defined as public.
Application-Based Case	In an **Order Processing System**, an `OrderProcessor` class (publisher) raises an `OrderPlaced` event when a new order is confirmed. A separate

	`EmailService` class (subscriber) subscribes to this event to automatically send a confirmation email.

4. Generics

Category	Description
Definition	A template mechanism that allows defining type-safe data structures, classes, and methods without committing to a specific data type at compile time.
Features	**- Type Safety:** Ensures correctness at compile time, reducing runtime errors. **- Performance Improvement:** Eliminates the need for boxing and unboxing of value types to object types, thus improving the overall performance. - Supports generic classes, fields, and methods.
Real-Life Example	The entire .NET Collections Framework, including `List<T>`, `Dictionary<TKey, TValue>`, and `Stack<T>`, is built using generics.
FAQ in Interviews	Q: What are generics and why are they useful? Q: What are the advantages of using generics over non-generic types (like `ArrayList`)?
Best Practices	Use constraints (e.g., `where T : class` or `where T : IComparable`) to limit the types that can be used as arguments, which adds flexibility while preserving type safety.
Application-Based Case	Creating a reusable **Data Container (Box<T>)** that can store and retrieve any single data type. You can create `Box<int>` to store an integer and `Box<string>` to store a string without rewriting the class, ensuring no boxing overhead occurs.

5. Reflection

Category	Description
Definition	The ability of a C# program to inspect its own metadata (assemblies, modules, types) at runtime. This means "reflection lets our program to look into ourself".
Features	**- Runtime Inspection:** View class properties and methods at runtime.

	- Dynamic Invocation: Invoke methods or create objects dynamically (late binding). **- Framework Building:** Used to build complex frameworks like Dependency Injection (DI), ORMs (e.g., Entity Framework), Unit Testing frameworks (e.g., NUnit, xUnit), and serializers (JSON/XML converters).
Real-Life Example	The ASP.NET Core framework uses reflection to scan assemblies and find all classes that implement a specific interface to register them for Dependency Injection.
FAQ in Interviews	Q: How does reflection work in C#? Q: What are the limitations of reflection?
Best Practices	Use reflection sparingly and only when dynamic behavior is necessary, as it is generally **slower and more complex to debug** than direct method calls. Be mindful of security risks, as it can be used to access private members (breaking encapsulation).
Application-Based Case	A custom **Object-Relational Mapper (ORM)** framework uses reflection to map properties of a C# class to columns in a database table. The framework dynamically inspects the class to determine property names and types to build SQL queries.

6. Threading

Category	**Description**
Definition	A mechanism to achieve **parallel execution** by running multiple blocks of code (threads/tasks) concurrently, usually to improve application responsiveness and resource utilization.
Features	- Allows concurrent execution of operations. - Modern C# uses the Task Parallel Library (TPL) for simplified task management. - Involves synchronization mechanisms like locks or `SemaphoreSlim` to manage shared resources and prevent race conditions.
Real-Life Example	In a **gaming application**, one thread handles the main game loop and rendering (UI), while another background thread manages asset loading, network communication, or physics calculations.
FAQ in Interviews	Q: Explain how threading works in C#.

	Q: How would you manage resource throttling in a custom thread pool using `Thread`, `Task`, and `SemaphoreSlim`?
Best Practices	Prefer using `Task` and the modern `async/await` pattern (see section 8) over directly manipulating raw `Thread` objects to simplify concurrency control and exception handling.
Application-Based Case	In a **Reporting Service**, the system launches multiple background tasks to generate various large reports simultaneously. By using multiple threads, the total report generation time is significantly reduced compared to running them sequentially.

7. Ref and Out Keywords

Category	**Description**
Definition	Keywords used to pass method arguments by reference instead of by value, allowing modifications inside the method to affect the original variable in the calling scope.
Features	**- ref:** The argument **must be initialized** before it is passed to the method. **- out:** The argument **must be assigned** a value inside the method before the method returns.
Real-Life Example	The `Int32.TryParse(string s, out int result)` method uses `out` to return the parsed integer while the method itself returns a `bool` to indicate success or failure.
FAQ in Interviews	Q: Explain the difference between the `ref` and `out` keywords. Q: Can you pass an uninitialized variable to a method using `ref`? (Answer: No)
Best Practices	Use `out` when you need a method to return multiple results. Use `ref` only when the method needs to both read and modify the existing value of the variable.
Application-Based Case	A **Financial Transaction System** needs a method `PerformExchange` that takes an initial `balance` (`ref` keyword) to update it directly, and simultaneously returns a status message (`out` keyword) detailing the result of the exchange.

8. Async & Await

Category	Description
Definition	Keywords used for non-blocking asynchronous programming, primarily designed to improve application responsiveness and scalability during I/O-bound operations (e.g., network calls, file access).
Features	**- async:** Marks a method as asynchronous, enabling the use of the `await` operator inside it. **- await:** Suspends the execution of the calling method, yielding control back to the caller's thread until the awaited operation is complete, thus preventing blocking. - Modern C# 8.0 supports Async Streams (`await foreach`) for asynchronous enumeration of data.
Real-Life Example	In an **ASP.NET Web API**, using `async` and `await` for database queries allows the server thread to handle hundreds of other client requests while waiting for the database response, dramatically increasing the API's concurrency and scalability.
FAQ in Interviews	Q: What is the difference between `async` and `await`? Q: Demonstrate how `async/await` can be misused (e.g., for CPU-bound tasks).
Best Practices	Use `async` and `await` "all the way" up the call stack to avoid synchronous blocking. Never use `async` methods for CPU-bound tasks without explicitly offloading the work to a background thread pool.
Application-Based Case	A **Data Dashboard** application uses `await FetchDataFromAPIAsync()` when loading data. This ensures the user interface (UI) remains completely responsive (no freezing) while the application waits for the remote server to return the data.